

공세민

Se Min Kong

로봇·소프트웨어 개발

센서 입력, 제어 명령, 실제 장비를 연결하는
소프트웨어를 만듭니다.

숭실대학교 소프트웨어학부를 졸업하고
SSAFY Robotics Track에서 공부하고 있습니다.



연락처

semin1224@gmail.com

github.com/SeMinKong

웹 포트폴리오

교육

SSAFY Robotics Track
2026.01 - 현재

자격

정보처리기사

국가기술자격 · 과학기술정보통신부
2026.09.11 취득

OPIc English IH

ACTFL · Intermediate High
2027.10.04까지 유효

학력

숭실대학교 소프트웨어학부 · 2020.03 - 2026.02

관심 분야

Computer Vision · Robotics / Physical AI

수상

SSAFY 공통 프로젝트 우수상

2026.08.10 · THING

숭실 캡스톤디자인 경진대회 장려상

2025.10.01 · Briefit

IT대학 소프트웨어 공모전 금상

2025.08.18 · Briefit

IT 프로젝트 프로리그 장려상

2025.11.22 · Briefit

거주지

Suwon, Republic of Korea

웹 포트폴리오

온라인 이력서

프로젝트

구현 경험

카메라 입력과 로봇 제어를 연결한 시스템 구현

ROS 2와 웹 서버 간 비동기 데이터 연동

다축 모터 제어와 텐던 구동계 통합

학습·추론을 위한 데이터 변환 파이프라인 구축

서버 기반 실시간 물리·게임 상태 관리

영역별 LLM 대화와 설계 문서 생성

기술 스택

Robotics · Simulation	Languages	AI · Vision	LLM · Backend	Platform · Collaboration
ROS 2	C++	PyTorch	FastAPI	Ubuntu
Isaac Sim	Python	YOLO	LangChain	Docker
Isaac Lab			Ollama	Git
			llama.cpp	Jira
			vLLM	

THING

기간 2026.07 - 08 · 6인 팀

역할 손 구조 모델링 · 모터 · ROS 연동

Blender · DYNAMIXEL · U2D2

ROS 2 · Cyclone DDS



캔 파지 · 팀 시연

7축 텐던 로봇 핸드

손의 21개 landmark를 7축 목표로 바꿔 구동하는 텐던 로봇 핸드입니다.

직접 맡은 구현

- Codex의 Blender 스킬을 활용해 인체 손의 관절 구조를 모방한 모델을 설계하고, 3D 프린팅으로 제작했습니다.
- DYNAMIXEL 모터 7개의 제어값을 조정하고, 동기 구동과 키보드 기반 조작 기능을 구현했습니다.
- U2D2 기반 모터 통신 모듈을 구성하고, 제어 노드에서 전달한 데이터를 모터 명령으로 변환하는 ROS 브릿지를 구현했습니다.

트러블슈팅

모터 제어 범위 조정 · DYNAMIXEL SDK의 전류 제한, 토크 설정, 회전 범위, 제어 모드별 동작을 확인했습니다. 각 파라미터가 구동에 미치는 영향을 비교하며 로봇손의 기구 조건에 맞게 제어값을 조정했습니다.

텐던 구동과 복귀 구조 · 손가락을 굽히는 동작과 원위치로 복귀하는 동작을 함께 고려해야 했습니다. 관절의 힘 작용점을 계산하고 텐던 배치를 검토해, 장력을 이용하는 복귀 방식으로 구조를 변경했습니다.

ROS 통신 주기 개선 · 제어 명령의 전달 주기를 개선하기 위해 ROS 통신 설정을 조정하고 Cyclone DDS를 활용했습니다. 모터 구동과 데이터 전달을 함께 점검하며 실시간 제어를 위한 통신 환경을 구성했습니다.

회고

Physical AI를 위한 시뮬레이션 학습까지 진행하지는 못했지만, 로봇의 기구와 모터 제어를 연결하는 기초를 익혔습니다. 제작 과정에서 통신 오류와 하드웨어 한계를 경험하며 소프트웨어만으로 해결하기 어려운 조건들을 이해했습니다. 이를 바탕으로 기구·통신 구조를 개선하고, 이후 시뮬레이션 학습과 실제 제어를 연결할 프로젝트를 구상하고 있습니다.

🏆 SSAFY 공통 프로젝트 우수상

README

기술 문서

프로젝트 시연

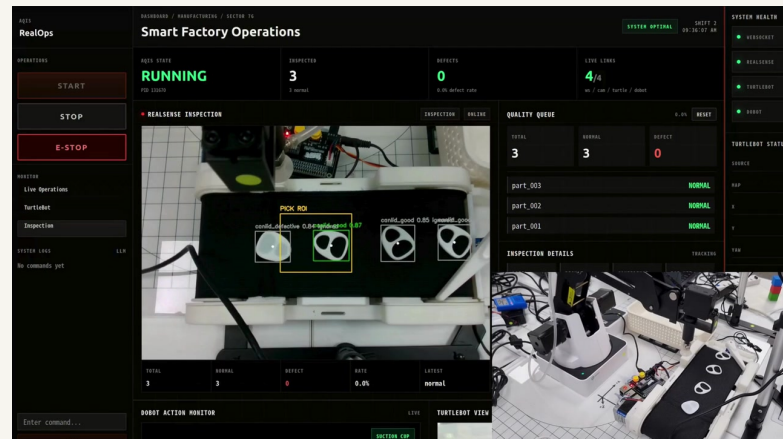
AQIS for Smart Factory

기간 2026.05 - 06 · 2인 팀

역할 팀장 · ROS 통합 · 비전 · 웹

ROS 2 · FastAPI · React

RealSense · YOLOv5 · Dobot



RealOps · 장비 관제

비전 검사·로봇 분류·웹 관제

불량 뚜껑 검출·로봇 분류·자율주행·웹 관제를 연결한 스마트팩토리입니다.

직접 맡은 구현

- Dobot, RealSense SDK, TurtleBot3, 비전·컨베이어 제어 패키지를 연결해 불량 캔 뚜껑을 검출하고 분류하는 통합 ROS 환경을 개발했습니다.
- 뚜껑 이미지 약 1,000장을 학습·검증 데이터로 7:3 분할하고, YOLO 객체 검출 모델을 학습했습니다.
- 각 장비의 동작 상태와 카메라 영상을 확인하는 모니터링 웹을 구현했습니다. 상태 데이터와 영상 전송 경로를 분리하고, 별도의 MJPEG 서버로 영상을 제공했습니다.
- ROS 내비게이션 패키지를 활용해 SLAM으로 지도를 구성하고, TurtleBot3가 지정 위치로 자율주행할 수 있도록 설정했습니다.

트러블슈팅

영상 전송 병목 · 영상과 상태 데이터를 WebSocket으로 함께 처리하면서 수신 주기가 불안정해졌습니다. TurtleBot과 RealSense의 영상을 별도 포트의 MJPEG 서버로 전달하고, 웹에서는 스트림을 표시하도록 변경했습니다. 그 결과 영상 갱신 속도를 약 4 FPS에서 30 FPS로 개선했습니다.

시뮬레이션·실물 연동 · 시뮬레이션과 실물 장비의 동작을 직접 동기화하는 과정에서 통합 난도가 높아졌습니다. 실행 환경을 분리하고 공통 데이터와 명령을 사용하도록 구성해, 각 환경에서 같은 공정 흐름을 수행할 수 있도록 했습니다.

이동 물체의 집기 좌표 · 고정 좌표로 집는 방식은 물체의 위치가 달라지면 대응하기 어려웠습니다. 검출한 위치를 집기 목표에 반영하고, 1초 간격으로 추론해 좌표를 갱신하도록 구성했습니다. 검출 시점과 집기 시점의 차이를 줄이는 데 초점을 맞췄습니다.

회고

여러 장비를 하나의 공정으로 연결하며 영상 처리, 통신, 로봇 동작의 주기를 함께 설계해야 한다는 점을 배웠습니다. 객체 검출과 정해진 제어 규칙을 연결한 구조에서 더 나아가, VLM의 장면 이해를 로봇의 행동 계획에 활용하는 방식도 탐색하고 싶습니다. 자율주행 역시 기존 패키지를 활용한 경험을 바탕으로 경로 계획과 제어를 직접 구현하고, 환경에 맞는 센서 데이터와 주행 조건을 다뤄보고 싶습니다.

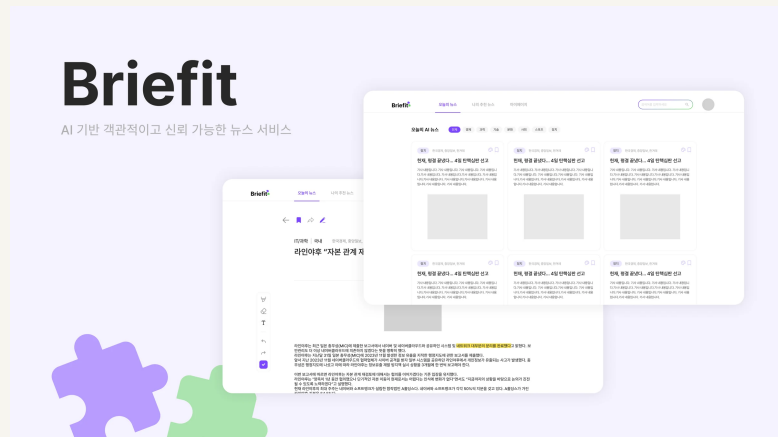
Briefit

기간 2025.05 - 09 · 6인 팀 · AI 2인

역할 기사 수집 · 요약 모델 · 평가

Python · Crawl4AI · KoBART

T5 · ROUGE · Gemini API



뉴스 목록 · 팀 서비스

뉴스 수집·요약 서비스

뉴스를 수집·그룹화·요약하는 팀 서비스의 2025년 AI 구현 경험입니다.

직접 맡은 구현

- Crawl4AI를 활용해 네이버 뉴스 약 4,000건을 수집했습니다. 기사 본문을 직접 확보하고 모델 입력에 맞게 정제하기 위해 크롤링 방식을 선택했습니다.
- 기사와 기준 요약으로 학습 데이터를 구성하고, 특정 카테고리에 맞는 요약을 생성하도록 KoBART 파인튜닝을 진행했습니다.
- T5와 KoBART 등 요약 모델의 출력을 ROUGE-1·ROUGE-2로 비교하고 분석했습니다.
- 기사 URL과 댓글 URL을 구분하고, 본문의 불필요한 태그와 지나치게 짧은 내용을 제외해 학습에 사용할 데이터를 정리했습니다.

트러블슈팅

파인튜닝 품질 한계 · 약 3,000건의 기사로 KoBART를 파인튜닝했지만 실사용에 필요한 요약 품질을 확보하기 어려웠습니다. 서비스 운영을 이어갈 수 있도록 Gemini API를 추가해 요약 기능을 보완했습니다.

장문 입력 처리 · 모델의 입력 길이를 넘는 기사는 문단 묶음으로 나눠 부분 요약한 뒤, 이를 합쳐 재요약하도록 구현했습니다. 다만 긴 단일 문단과 재요약 입력에서는 일부 내용이 잘릴 수 있어 문맥 보존에 한계가 남았습니다.

반복 문장 후처리 · 요약 끝의 반복 문장과 짧은 문장을 규칙으로 정리하는 후처리를 구현했습니다. 정상적인 짧은 문장도 제거될 수 있어, 반복 제거와 정보 보존을 함께 고려해야 하는 방식이었습니다.

회고

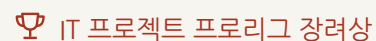
모델 구조 학습, 서비스 운영, 팀 협업을 동시에 진행하면서 충분한 실험과 품질 검토를 병행하기 어려웠습니다. 이 과정에서 Transformer 기반 요약 모델의 학습·생성 흐름을 이해했고, 모델 평가 점수와 사용자가 체감하는 품질 사이에 차이가 있다는 점을 배웠습니다. 이후에는 팀 내 요구사항과 품질 기준을 먼저 맞추고, 실제 기사와 사용자 경험을 기준으로 개선 과정을 쌓아가고 싶습니다.



SW 공모전 금상



캡스톤디자인 장려상



IT 프로젝트 프로리그 장려상

README

기술 문서

프로젝트 시연

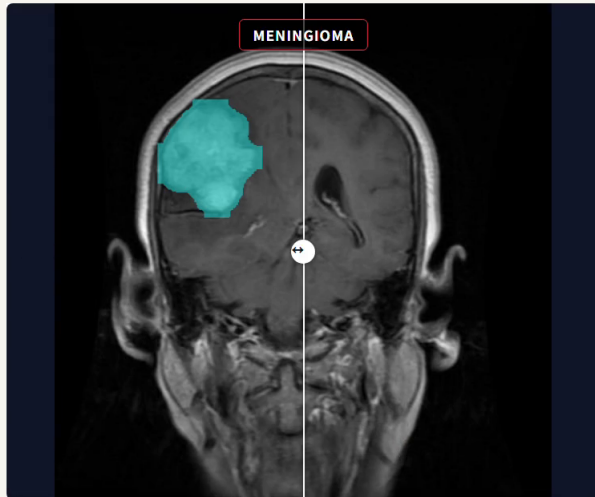
Brain Tumor MRI

기간 2026.02 - 04 · 개인 프로젝트

역할 전처리 · 학습 · 추론 · 웹 데모

Python · PyTorch

Ultralytics YOLO11 · OpenCV



분류·분할 통합 추론

MRI 분류·종양 영역 시각화

MRI의 종양 유형과 영역을 독립된 두 모델로 분석하고 함께 시각화합니다.

직접 맡은 구현

- MRI의 종양 유형을 분류하는 모델과 종양 영역을 분할하는 모델을 YOLO11 기반으로 구성하고, 각각의 학습·가중치 저장 경로를 구현했습니다.
- 픽셀 마스크에 이진화와 형태학 연산을 적용하고, 외부 윤곽을 추출해 YOLO 분할 학습용 다각형 라벨로 변환했습니다.
- 같은 MRI에 두 모델을 적용하고, 분류 범주·신뢰도와 분할 영역을 한 이미지에서 비교할 수 있는 통합 추론·시각화 기능을 구현했습니다.

트러블슈팅

마스크 윤곽 정제 · 마스크의 작은 틈과 돌기가 다각형 라벨에 반영될 수 있어 closing·opening 연산과 점수·면적 필터를 적용했습니다. 다만 작은 종양 영역까지 제거될 수 있어 변환 전후의 라벨을 비교해야 하는 한계가 남았습니다.

모델 출력 비교 · 분류와 분할은 독립적으로 동작하므로 서로 다른 결과를 낼 수 있습니다. 같은 입력의 분류 범주·신뢰도와 분할 영역을 함께 표시해, 결과의 차이를 확인할 수 있도록 구성했습니다.

회고

분류와 분할을 함께 구현하며 모델 선택뿐 아니라 정답 라벨을 어떤 형태로 변환하는지도 결과에 영향을 준다는 점을 배웠습니다. 특히 전처리에서 작은 영역이 사라지는 문제와 평가 데이터의 분리 기준을 더 면밀하게 다뤄야 했습니다. 다음에는 학습·모델 선택에 사용하지 않은 최종 평가셋을 확보하고, 실행별 가중치와 로그를 남겨 결과를 비교하고 싶습니다.

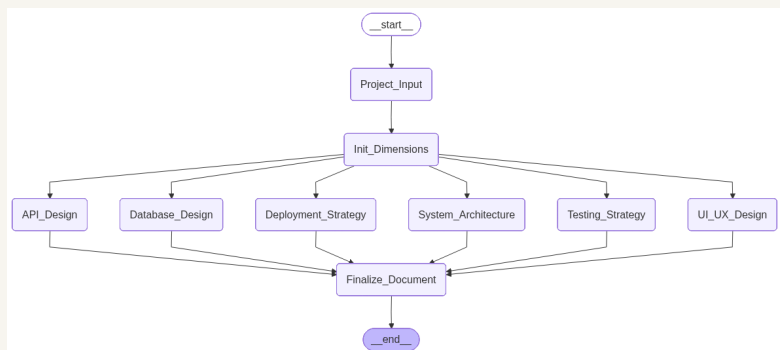
Project Prompt Generator

기간 2026 · 개인 프로젝트

역할 대화 서버 · 상태 관리 · 웹 UI

Python · FastAPI · WebSocket

LangChain · Solar Pro



영역별 대화 · 초기 설계

영역별 대화·설계 문서 생성

여섯 영역의 대화를 분리하고 결과를 Markdown 설계 문서로 생성합니다.

직접 맡은 구현

- UI/UX, 시스템 구조, 데이터베이스, API, 배포, 테스트의 여섯 영역으로 대화를 나누고, 영역별 이력·진행 상태·생성 결과를 독립적으로 관리했습니다.
- 프로젝트 설명과 해당 영역의 대화 이력을 조합해 LLM 입력을 구성하고, 초기 질문의 병렬 호출과 동기 LLM 작업의 별도 스레드 처리를 구현했습니다.
- 사용자가 영역별 결과를 추가 대화로 수정하고, 저장된 결과를 모아 하나의 Markdown 설계 문서로 생성하는 흐름을 구현했습니다.

트러블슈팅

생성 완료 판정 · LLM이 응답했다고 해서 설계 결과가 완성된 것은 아니므로, 대화 진행과 결과 생성을 구분했습니다. 3라운드 이상 진행되고 생성 태그가 포함된 경우에만 완료로 처리하며, 조건을 충족하지 않으면 대화를 이어가도록 구성했습니다.

실패 턴 복원 · LLM 호출 전에 증가시킨 라운드가 실패 후에도 남지 않도록, 처리 대상 오류에서 이전 값으로 복원했습니다. 대화 이력은 호출 성공 후 기록하고, 수정 중 오류가 발생해도 이전 생성 결과는 유지하도록 했습니다.

회고

LLM을 활용한 도구에서는 프롬프트 작성과 함께 문맥의 범위, 대화 상태, 완료 조건을 설계하는 일이 중요하다는 점을 배웠습니다. 영역별로 대화를 분리하면서 사용자가 특정 부분만 수정할 수 있는 구조도 경험했습니다. 다음에는 연결 종료 후에도 작업을 이어갈 수 있는 복구 기능과, 생성 문서가 요구사항을 얼마나 반영했는지 평가하는 기준을 보완하고 싶습니다.

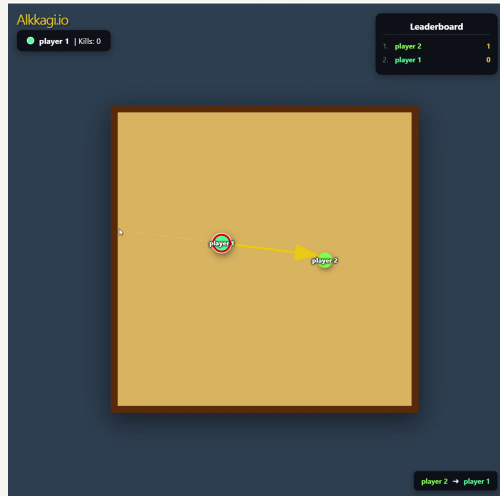
Alkkagi.io

기간 2026.03 - 04 · 개인 프로젝트

역할 웹 UI · 통신 · 서버 물리

React · TypeScript · Node.js

Express · Socket.IO



드래그 조준 · 실제 플레이

실시간 멀티플레이 알까기

서버가 물리·득점을 계산해 공통 상태를 전달하는 실시간 웹 대전 게임입니다.

직접 맡은 구현

- React와 Socket.IO로 조준 입력과 게임 상태를 연결하고, 서버에서 이동·충돌·득점을 계산해 모든 플레이어에게 공통 상태를 전달하도록 구현했습니다.
- 돌의 겹침을 해소하는 위치 보정과 질량·반발계수를 반영한 충격량 계산을 분리해 충돌 처리를 구현했습니다.
- 한 번의 상태 갱신을 10개 소단계로 나눠 이동·마찰·충돌을 계산하고, 보드 이탈에 따른 점수 이전·돌의 성장·재배치를 같은 흐름에 통합했습니다.

트러블슈팅

겹침과 충돌 반응 분리 · 돌이 겹쳐 있어도 이미 서로 멀어지고 있다면 추가 충격량을 적용할 필요가 없습니다. 겹친 위치를 먼저 보정하고, 충돌 방향의 상대 속도를 확인해 접근하는 경우에만 충격량을 적용하도록 구성했습니다.

소단계 감속 보정 · 물리 계산을 10번으로 나누면서 각 단계에 같은 마찰계수를 적용하면 감속이 과해집니다. 단계별 계수를 $0.8 * 0.1$ 로 보정해, 다른 충돌·정지·재배치 처리가 없는 경우 전체 10단계의 감속이 기존의 0.8배를 유지하도록 했습니다.

회고

물리 계산을 직접 구현하며 충돌 수식뿐 아니라 시간 간격과 처리 순서가 움직임에 영향을 준다는 점을 배웠습니다. 또한 여러 사용자가 같은 게임을 플레이하려면 판정의 기준 상태를 어디서 관리할지도 함께 설계해야 했습니다. 다음에는 고속 충돌과 다중 접촉을 반복 검증하고, 네트워크 지연과 동시 접속 부하가 플레이에 미치는 영향을 측정하고 싶습니다.

프로젝트 자료

README · 기술 문서 · 시연

README

개요 · 역할 · 실행 방법

기술 문서

코드 · 계산식 · 검증 범위

프로젝트 시연

영상 · 실제 화면 · 결과 자료

THING

README 기술 문서

AQIS for Smart Factory

README 기술 문서

Briefit

README 기술 문서

Brain Tumor MRI

README 기술 문서

Project Prompt Generator

README 기술 문서

Alkkagi.io

README 기술 문서



웹 포트폴리오

semin1224@gmail.com

github.com/SeMinKong

프로젝트

이력서 · 수상